DAG-Based Workflow Orchestrator

Design Decisions & Trade-off Summary

Problem 2 · Backend Engineering Assignment · Node.js + TypeScript

This document captures the 12 key architectural decisions for Problem 2 of the take-home assignment. Each decision records the chosen approach, the rejected alternatives, and the engineering rationale. Decisions were finalized after an adversarial review session and are considered locked for implementation.

1 Infrastructure & Persistence

#1 Persistence Layer

✓ **CHOSEN: PostgreSQL**

✗ MySQL 8+ — gap-lock behavior is quirky; SKIP LOCKED semantics less predictable

*SELECT ... FOR UPDATE SKIP LOCKED gives us a clean, lock-free task-claiming primitive out of the box. Postgres also provides atomic counter updates via RETURNING *, which is exploited directly in the ready-set scheduler (Decision #3). No custom locking code needed.*

```
SELECT * FROM tasks WHERE status='READY' ORDER BY priority LIMIT 1 FOR UPDATE SKIP LOCKED;
```

#2 Worker Architecture

✓ **CHOSEN: Single-process with bounded semaphore (counter + waiter queue)**

✗ Multi-process — requires distributed lease/heartbeat coordination, out of scope for 3-day budget

✗ p-limit / async-pool / bottleneck — forbidden by spec; would hide concurrency mechanics

Single-process satisfies the spec and all listed deliverables (500-task diamond DAG, kill -9 recovery, retry storms). The bounded semaphore is implemented from scratch: an integer counter and a Promise-based waiter queue. Multi-process design is described in the design doc as a production evolution path.

2 DAG Topology & Scheduling

#3 Cycle Detection Algorithm

✓ **CHOSEN: DFS Three-Coloring (WHITE / GRAY / BLACK)**

✗ Kahn's algorithm (BFS topological sort) — correct but only returns a boolean; no cycle path

Three-coloring surfaces the exact back-edge that closes the cycle, enabling a rejection error message like 'Cycle detected: A → B → C → A'. This is superior UX on submission failure and is trivially testable against adversarial DAGs (self-loops, diamonds, disconnected components).

```
Colors: WHITE=unseen, GRAY=in-stack, BLACK=done. Back-edge to GRAY node = cycle.
```

#4 Ready-Set Atomicity (Parent Completion → Child Enqueue)

✓ **CHOSEN: DB-atomic UPDATE ... SET pending_deps = pending_deps - 1 ... RETURNING ***

✗ In-memory adjacency + mutex — faster but loses state on crash; recovery becomes complex

✗ Application-level read-modify-write — classic TOCTOU race when two parents complete concurrently

The critical race: two parent tasks complete concurrently and both decrement the same child's in-degree. A single Postgres UPDATE with RETURNING atomically decrements and returns the new value; the worker that sees pending_deps = 0 in the result enqueues the child. No double-enqueue possible. No mutex needed.

```
UPDATE tasks SET pending_deps = pending_deps - 1 WHERE id=$1 RETURNING pending_deps;
```

#5 In-Memory Priority Queue

✓ **CHOSEN: Min-heap keyed on (-priority, scheduled_time, submission_order)**

✗ Array.sort() per dequeue — O(n log n) per operation; forbidden by spec

✗ Sorted linked list — O(n) insert; fine for small N but not defensible

Binary min-heap gives O(log n) insert and O(log n) extract-min. Tie-breaking is deterministic via the compound key tuple. Heap sift-up and sift-down are implemented from scratch. The negative priority ensures higher numeric priority = earlier dequeue from a min-heap without a custom comparator inversion.

3 Reliability & Crash Recovery

#6 Crash Recovery — Task Persistence Granularity

✓ **CHOSEN: Strict: two DB writes per state transition + periodic heartbeat while RUNNING**

✗ Pragmatic (one write per transition) — simpler but cannot distinguish completed vs crashed RUNNING tasks without heartbeat anyway

✗ Heartbeat-only — insufficient; transition state lost between heartbeats

Write new state BEFORE handler execution (e.g., RUNNING), write final state AFTER (COMPLETED / FAILED). A process crash between writes leaves the task in RUNNING. The heartbeat reaper reclaims tasks where now - last_heartbeat_at exceeds the lease timeout and re-enqueues them, respecting retry policy. Strict writes are cheap on Postgres at 500-task scale.

```
Schema: tasks.last_heartbeat_at TIMESTAMPTZ; reaper polls every ~10s.
```

#7 Retry Backoff Strategy

✓ **CHOSEN: Decorrelated jitter: sleep = min(cap, random(base, prev_sleep * 3))**

✗ Full jitter: random(0, min(cap, base * 2^attempt)) — good spread but occasionally retries near-instantly

✗ Equal jitter: half + random(0, half) — bounded floor, less spread, no self-adaptation

✗ Fixed exponential — no jitter; thundering herd on correlated failures

Decorrelated jitter self-adapts: if a previous retry was short, the next range is narrow; if long, the range widens. This is the AWS architecture blog recommendation for avoiding retry storms (directly tested by the 100-task x 5-failure deliverable). Cost: one extra column (last_sleep_ms) in the tasks table.

```
next_sleep = min(cap, random(base, prev_sleep * 3)) // all in milliseconds
```

#8 Task Timeout Enforcement

✓ **CHOSEN: Hard timeout via Promise.race + AbortSignal, failure counted against retry policy**

✗ Soft timeout (log + continue) — defeats the purpose of timeoutMs; not defensible

✗ Process-level kill — can't target a single task in single-process model

Promise.race between the handler promise and a setTimeout-backed rejection. AbortSignal is passed to the handler so it can propagate cancellation to downstream I/O (fetch, pg queries). On timeout, the attempt is counted as FAILED and retry policy is evaluated normally.

```
await Promise.race([handler(input, { signal }), timeoutReject(timeoutMs)]);
```

4 Workflow Semantics & Features

#9 Cancel Semantics

✓ **CHOSEN: Cooperative cancel: mark workflow CANCELLING; running tasks complete; dependents never enqueued**

✗ Preemptive cancel (abort in-flight handlers) — requires handlers to be abort-aware; complex contract

✗ Immediate CANCELLED + rollback — no rollback primitive defined; risks partial side effects

Cooperative cancel is safe and auditable. The state machine is: CANCELLING → (all running tasks drain) → CANCELLED. No new tasks are dequeued from a CANCELLING workflow. The worker checks workflow status before executing any READY task.

#10 Idempotency Primitives

✓ **CHOSEN: Expose taskRunId (attempt-scoped UUID) as context.idempotencyKey to every handler invocation**

✗ Framework-managed idempotency store — over-engineering; framework cannot know handler's external targets

✗ No primitive — leaves handler authors with no dedup anchor

The framework guarantees at-least-once execution. Exactly-once-effective is the handler author's responsibility, achieved by using context.idempotencyKey as a unique constraint key in a DB upsert or as an idempotency header in an external API call. This is documented as the canonical pattern; no additional framework code required.

```
handler(input, { idempotencyKey, signal, workflowId, taskId, attempt })
```

#11 Cron Timezone Strategy

✓ **CHOSEN: Timezone-aware: IANA zone per schedule (default UTC); skip DST gap; fire once on DST fold (first occurrence)**

✗ UTC-only — eliminates DST boundaries entirely; evaluators specifically probe DST correctness

The spec explicitly states 'Cron parser correctness across DST boundaries — pick a timezone strategy and defend it.' UTC-only is not a strategy; it's avoidance. Timezone-aware adds ~40 lines using Intl.DateTimeFormat to resolve UTC offsets. DST gap rule: if next-fire falls in a skipped hour, advance to post-gap. DST fold rule: if next-fire falls in a repeated hour, use first occurrence.

```
schedules.timezone VARCHAR DEFAULT 'UTC'; // IANA e.g. 'America/New_York'
```

#12 Cron Schedule — Workflow Definition Storage

✓ **CHOSEN: Inline: cron schedule record stores full workflow definition as a JSONB column**

✗ Pre-registered named templates (POST /templates endpoint) — extra table, extra endpoint, template versioning questions; out of scope for 3-day budget

On each cron fire, the scheduler deep-clones the inline JSON and submits it to the same POST /workflows code path. No additional abstraction layer. Named templates are described in the design doc as a natural v2 evolution (POST /templates → cron schedule stores template_id + overrides).

```
schedules.workflow_definition JSONB NOT NULL;
```

5 Decision Summary

#	Decision	Chosen
1	Persistence Layer	**PostgreSQL**
2	Worker Architecture	**Single-process + bounded semaphore**
3	Cycle Detection	**DFS Three-Coloring**
4	Ready-Set Atomicity	**DB-atomic UPDATE ... RETURNING**
5	Priority Queue	**Min-heap (scratch)**
6	Crash Recovery	**Strict 2-writes/transition + heartbeat reaper**
7	Retry Backoff	**Decorrelated jitter**
8	Task Timeout	**Promise.race + AbortSignal (hard)**
9	Cancel Semantics	**Cooperative (CANCELLING drain)**
10	Idempotency	**taskRunId exposed as idempotencyKey**
11	Cron Timezone	**Timezone-aware (IANA, skip gap, fold-first)**
12	Cron Workflow Def	**Inline JSONB in schedule record**